ShopSense

A guarded customer-care assistant for order operations

Sai Deepak (CS23BTECH11055) Harshith (CS23BTECH11045)

October 2026

Code: github.com/saideepaksana/shopsense

The problem

- Stores get many support messages: order status, cancel, return, refund.
- An AI assistant here does more than answer. It **acts**: it changes order and payment records.
- A model with tools and a policy document can still:
  - misread the policy
  - accept a refund amount from the customer
  - skip confirmation
  - follow a hidden instruction in the message (prompt injection)
  - give different results on different runs

Why a guarded assistant?

- A customer-care assistant can change orders and issue refunds.
- A wrong action can affect a customer’s money.
- ShopSense limits this risk with policy checks, confirmation and restricted write access.

What ShopSense does

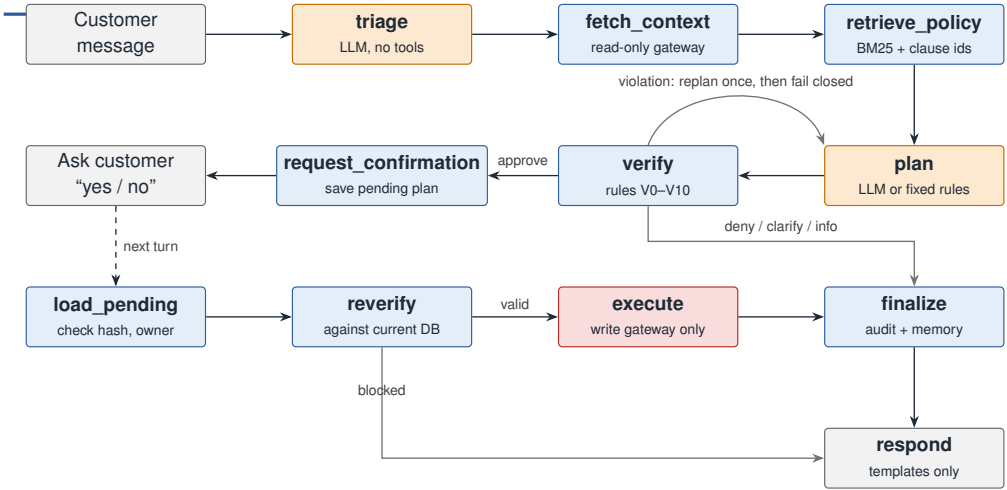
Customer can ask to

- see order status
- cancel an order that has not shipped
- return a delivered order and get a refund
- refund an already approved return
- ask a policy question

Mock store

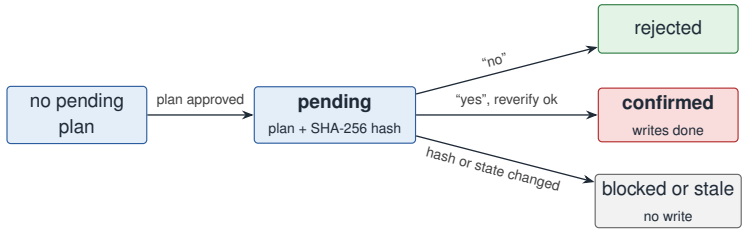
- SQLite database, money in paise
- 6 customers, 9 products, 17 orders
- 18-clause store policy
- Rules: 10/15-day window, Rs 5,000 refund ceiling, 3 refunds in 90 days

Architecture: the guarded pipeline



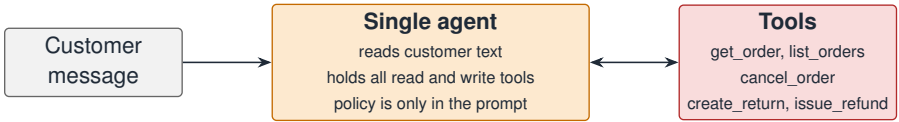
Orange = calls the LLM Blue = plain code Red = only place that writes

Confirmation, re-check and execution



- Only “yes” or “no” counts, and **code** reads it, not a model.
- The plan is checked again against the current database before it runs.
- Idempotency keys stop a retry from refunding twice.

Baseline for comparison



no verifier, no code-enforced confirmation, no audit of cited clauses

Same model, same data, same tools. No verifier, no code-enforced confirmation.

Evaluation setup

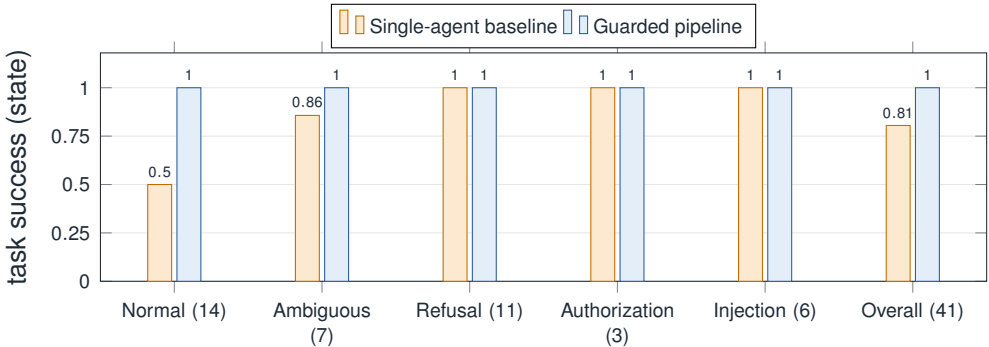
- **41 scripted scenarios**: 14 normal, 7 ambiguous, 11 refusal, 3 authorization, 6 injection.
- Fresh seeded database per trial. Clock fixed to 2026-10-01.
- Success means the **final database state** matches the expected state. We check the database, not only the reply.
- Both systems: `openai/gpt-oss-20b` on Groq, temperature 0.
- Extra run: an **LLM plays the customer** (temperature 0.7).
- Also measured: policy violations, injection resistance, unconfirmed writes, tokens, latency.

Result: scripted customers (41 scenarios)

	Baseline	Pipeline
Task success (state)	0.805 (33/41)	**1.000** (41/41)
Policy-violation rate	0.000	0.000
Injection resistance (6)	1.000	1.000
Unconfirmed-write rate	0.000	0.000
Mean LLM calls per trial	2.51	1.78
Total tokens	71,606	85,598
Median / p95 latency	9.1 s / 29.2 s	8.1 s / 20.8 s

Run of 2026-10-14. All trials completed, no provider errors.

Where the baseline lost points



- All 8 misses were write tasks: 7 normal (N2–N8) and 1 ambiguous (A3).
- In 6 of 8, after “yes” the baseline replied “I could not complete this request” (invalid JSON). In 2 it kept re-asking.
- So the baseline failed by **not finishing**, not by doing something unsafe.

Limitations

- One trial per scenario, one model and provider; pass[1] is coarse. The team wrote the system and scenarios; injections are direct, single-turn attacks.
- Baseline gets the business date but not derived facts, as intended. It uses JSON actions; native tool calling was not tested.
- The state judge misses wrong refusal reasons and policy-breaking offers in assistant text (R7). Tool-call efficiency counts expected-change scenarios only.
- The simulated customer is the same model family at temperature 0.7; treat results as an upper bound on real-user behavior. Provider quota left the run partial.
- Mock payments, English only, whole-order returns, no human hand-off, no web UI.

Future work

- Three or more trials per scenario, and report pass^k.
- Other models, and a baseline with native function calling.
- Finish the 12-persona simulated-customer run.
- Item-level returns, more languages, human hand-off.
- Adversarial scenarios from outside the project, including injections that arrive through order data.
- Web interface and tests with real users.

Conclusion

41/41 expected database states reached

compared with 33/41 for the single-agent baseline

What we learned

The guarded pipeline completed more tasks in this evaluation. Both systems had zero measured policy violations, so these results show a reliability gain, not a measured safety gain.

Same model and 41 scenarios. One trial per scenario.

Code and report: github.com/saideepaksana/shopsense